Separating memory capacity from data movement in neural network inference

Geunsik Lim[1*]

[1] College of Information and Communication Engineering, Sungkyunkwan University (SKKU), Suwon, South Korea .

Corresponding author(s). E-mail(s): leemgs@g.skku.edu;

Abstract

Running large neural networks now depends less on arithmetic speed than on how efficiently data move between fast and slow memory, and inference dominates the recurring cost and energy of deployed models [1–3]. Many techniques relieve this pressure, including offloading, swapping, caching and compression [4–7], yet their reported gains are hard to compare. The operating condition of each measurement usually goes unreported, and common bandwidth measures reuse the very latency they are meant to explain. Here we introduce a measurement framework that describes any operating point with two independently measurable ratios: how much of the active data can reside in fast memory, and how long computation takes relative to the transfers it must hide. From a conservative bound on service time we derive two floors that no placement or scheduling policy can remove, together with a minimal set of reported quantities that lets any reader recompute both ratios. Small probes on a server processor and on accelerators from two vendors show that the protocol is executable and that latency rises in the predicted direction as residency falls. The framework gives studies of memory-bound inference a common, auditable coordinate system; whether it predicts the fastest orchestration policy remains an open, preregistered test.

When model weights, activations and key–value (KV) caches exceed accelerator memory, inference latency is governed less by arithmetic than by how execution coordinates data movement across device memory, host memory and storage [1, 2]. Because inference dominates the recurring cost and energy of deployed models, this coordination has direct economic and environmental stakes [3, 8, 9], and it recurs across autoregressive

language models [10, 11], vision and vision–language models [12, 13] and retrieval-augmented pipelines [14]. Hardware analyses identify memory capacity, bandwidth and interconnect latency as governing constraints [15–17].

Systems relieve this pressure along many axes. Serving frameworks manage the KV cache and schedule requests [4, 18–20]; offloading and memory-pooling systems spill weights or activations to host memory or storage [5, 21–25]; swapping systems stage tensors on and off the accelerator [6, 26, 27]; sparsity, pruning and IO-aware attention shrink the working set or its traffic [7, 28, 29]; parallelism redistributes memory across devices [30, 31]; and host allocators and NUMA-aware placement shape the sustained transfer path [32–35]. The design space has prompted several surveys [36, 37] and widens further on edge and heterogeneous hardware [38–41]. Yet the same technique can help or hurt depending on model size, batch size, hardware topology and how much compulsory traffic remains on the critical path, and published comparisons rarely report enough to tell which condition applied.

Classical tools each capture one side of this problem. Working-set analysis [42] treats capacity saturation on a single tier, and the Roofline model [43] bounds performance by $\min(F \cdot I, B_{\text{peak}})$ for a fixed workload on a single tier. Neither alone distinguishes two operating points with the same arithmetic intensity and working-set size when one can overlap its cross-tier traffic and the other cannot, whereas queueing descriptions of overlapping services [44] represent overlap but supply no portable coordinate for residency. We address this narrow gap with a common coordinate system for a multi-tier hierarchy, not a replacement for these frameworks.

We make three contributions. First, an identifiable operating coordinate: independently measurable residency R_C and overlap R_B separate capacity pressure from exposed transfer without using total latency to explain itself. Second, bounds that map diagnosis to intervention: a conservative service-time bound yields a minimum non-resident share $\max(0, 1 - R_C)$ and an unhideable transfer share $\max(0, 1 - R_B)$; only $R_B = 1$ is a derived crossover, whereas $R_C = 0.5$ is a declared convention whose sensitivity must be reported. Third, a falsifiable open protocol with bounded evidence: a compiled processor probe and layered-matrix runs on 3 accelerators of two vendors test the predicted directions. The evidence establishes directional consistency, not production effect sizes, policy rankings or per-device boundary values.

A two-ratio operating coordinate

We describe a hierarchical-memory operating point using

$$R_C = \frac{C_{\text{fast}}}{W}, \qquad R_B = \frac{B_{\text{slow}} T_{\text{comp}}}{D} = \frac{T_{\text{comp}}}{T_{\text{transfer}}}, \tag{1}$$

where C_{fast} is usable fast-memory capacity, W is the active working set, D is compulsory cross-tier traffic per step, B_{slow} is sustained bandwidth on the limiting link, and $T_{\text{transfer}} = D/B_{\text{slow}}$. Unlike a ratio that uses total step time, R_B is not circular: it compares independently timed computation and transfer intervals.

We use $\theta_C = 0.50$ as an explicit majority-residency convention and derive $\theta_B = 1$ from equality of computation and transfer time, giving three operational labels

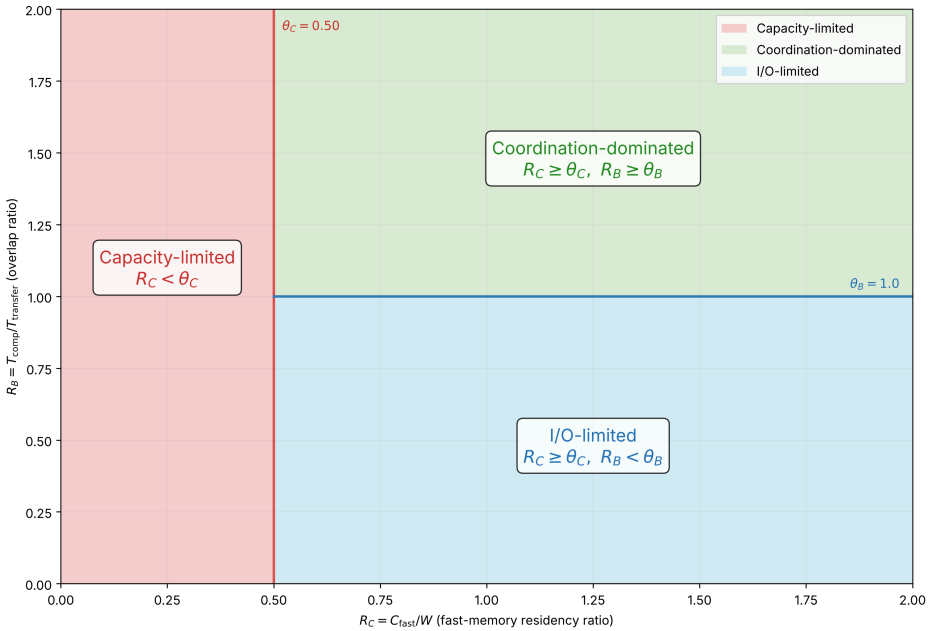


Fig. 1 Analytical regime map. The vertical line is the declared majority-residency convention $\theta_C = 0.50$; the horizontal segment is the derived overlap boundary $\theta_B = 1.0$. The diagram is generated from the same constants as the released classifier and contains no empirical data.

(Fig. 1): capacity-limited ($R_C < \theta_C$); I/O-limited ($R_C \geq \theta_C$ and $R_B < \theta_B$); and coordination-dominated ($R_C \geq \theta_C$ and $R_B \geq \theta_B$). These are screening labels, not causal bottleneck estimates: a "capacity-limited" point need not have capacity as its largest latency term, and continuous ratios with dimensional timings carry the evidence. The capacity boundary is a convention, not a critical point, whereas crossing $R_B = 1$ changes the sign of the ideal-overlap remainder: for $R_B < 1$, at least $T_{\text{transfer}} - T_{\text{comp}}$ cannot be hidden. Primary analyses therefore retain continuous (R_C, R_B), so that a convenient residency cut-off is not presented as an empirically discovered constant.

Irreducible costs and predictions

For accounting purposes, define non-overlapping critical-path contributions so that a step can be written as

$$T_{\text{total}} = T_{\text{comp}} + T_{\text{mem}} + T_{\text{swap}} + T_{\text{sync}}. \tag{2}$$

Independently of that decomposition, a conservative service-time lower bound that allows the underlying services to overlap is

$$T_{\text{total}} \geq \max\left\{T_{\text{comp}}, \rho D_{\text{nr}}, \frac{D}{B_{\text{slow}}}\right\}, \tag{3}$$

where D_{nr} is the number of non-resident bytes compulsorily accessed in the step and ρ is a lower bound on their service time per byte. R_C bounds storage residency but does not determine D_{nr} unless an access-coverage model is specified. Taking the maximum permits ideal overlap and avoids double-counting bytes that contribute to both a cache miss and a cross-tier transfer. Beyond a single-tier roofline bound $\max(T_{\mathrm{comp}}, D/B_{\mathrm{peak}})$, Equation (3) adds an explicit residency floor ρD_{nr} and uses sustained rather than peak bandwidth, so a capacity-limited operating point cannot be hidden by arithmetic intensity alone. Synchronisation and contention can only increase observed time, so the bound is not a performance predictor, and the residual above it cannot be assigned to one latency component without event-level measurements.

Two normalized consequences make the formulation useful beyond relabelling a roofline plot. Under the aggregate-capacity approximation, no placement policy can make more than C_{fast} bytes simultaneously resident, so the minimum non-resident storage share of an active working set is

$$f_{\mathrm{nonresident}} \geq \max(0, 1 - R_C). \tag{4}$$

Even a perfect overlap schedule leaves an exposed interval $\max(0, T_{\mathrm{transfer}} - T_{\mathrm{comp}})$; normalized by transfer time,

$$f_{\mathrm{exposed}} \geq \max(0, 1 - R_B). \tag{5}$$

Thus R_C answers how much placement pressure is irreducible, whereas R_B answers how much compulsory-transfer service cannot be hidden. Both are scale-invariant: multiplying capacity and working set, or both isolated service times, by the same factor leaves the ratio unchanged.

This separation gives an actionable diagnostic without asserting an optimal policy. If $R_C < 0.5$, interventions can reduce the active working set, increase usable capacity, or reduce the bytes touched while non-resident. If $R_C \geq 0.5$ but $R_B < 1$, overlap alone cannot remove the exposed interval, so reducing compulsory traffic or increasing sustained link bandwidth is necessary. If both tests pass, the residual directs measurement toward synchronisation, launch, contention or locality costs.

Both ratios are computable from datasheets before any run. For example, a 70B-parameter model holds roughly 140 GB of weights in `fp16`. On a single 40 GB accelerator, $R_C = 40/(140 + K) \leq 40/140 \approx 0.29 < 0.5$ for any KV/activation footprint $K \geq 0$, so no offloading schedule can remove the majority-eviction cost; four such accelerators ($C_{\mathrm{fast}} = 160$ GB) give $R_C \approx 1$. This is an illustrative calculation from published capacities and parameter counts, not an empirical measurement, and is exactly the kind of decision the two ratios make explicit before benchmarking.

The formulation makes two falsifiable predictions. First, reducing residency while holding the access pattern fixed should expose a higher memory-access cost when it increases D_{nr}. Second, when $R_C \geq \theta_C$, configurations with $R_B < 1$ must expose transfer time even under ideal compute–transfer overlap. It does not, without additional measurements, predict the magnitude of a policy's speed-up or which policy is fastest. The two coordinates are jointly necessary but not sufficient for latency prediction: equal (R_C, R_B) does not imply equal latency, and this non-identifiability defines

Table 1 Selected measured points from the compiled CPU pointer-chain experiment. Latency is mean ± one standard deviation over seven trials; the last column is a 95% percentile bootstrap confidence interval for the mean, quantifying within-machine trial variability (not a hardware-population interval; see Methods).

Working set (MiB)	R_C	Latency (ns/load)	95% CI (ns)	Interpretation
0.5	96.000	6.8 ± 0.1	[6.8, 6.9]	small working set
48	1.000	118.5 ± 6.6	[114.0, 123.7]	reported LLC capacity
96	0.500	132.5 ± 14.3	[124.5, 144.5]	residency convention
256	0.188	169.7 ± 10.4	[161.5, 176.9]	largest working set

the residual phenomena that must be measured rather than attributed to capacity or bandwidth.

Residency effects on a processor

We evaluated working-set sensitivity on an Intel Xeon Platinum 8370C processor in a virtualised Linux environment, using a randomised pointer chain over working sets from 0.5 to 256 MiB with seven trials per point (Methods). Linux sysfs reported a 48 MiB last-level cache, which we used as C_{fast}. Mean dependent-load time was 6.8 ± 0.1 ns at 0.5 MiB ($R_C = 96$), 118.5 ± 6.6 ns at 48 MiB ($R_C = 1$), 132.5 ± 14.3 ns at 96 MiB ($R_C = 0.5$), and 169.7 ± 10.4 ns at 256 MiB ($R_C = 0.188$; mean ± s.d. across trials; Table 1). The non-monotonic intermediate points show that cache level, translation, virtualisation and system noise matter, and the data show no discontinuity at the conventional $R_C = 0.5$ label. A separate layered-matrix harness on the same processor gave the same residency direction (Methods).

Residency and overlap on accelerators

We ran the layered program on 3 accelerators of distinct architectures and two vendors—an NVIDIA Tesla T4, an NVIDIA A100 and a Google TPU v5e—with $d = 2048$, 24 layers, batch size 8 and ten timing windows. On the CUDA devices, computation and host-to-device transfer were timed separately with CUDA events and non-resident layer weights were staged from pinned host memory each step, so R_C and R_B were varied by construction. Unlike a single processor's memory system, the device–host link makes the exposed-transfer regime physically reachable.

The residency prediction is directionally consistent on all 3 accelerators. On every device, mean per-step latency in the capacity-limited region ($R_C < \theta_C$) exceeded that at full residency ($R_C \geq 1$), by 1.23–2.83× (Table 2). The formulation predicts this *direction*, not its magnitude; the magnitude shrinks on higher-bandwidth devices, where the offload penalty is smaller relative to compute. Latency is monotone in the number of offloaded layers on all three devices except at the fully resident point on the A100 and the TPU, which we trace to a measurement artifact of its structurally distinct code path rather than to memory behaviour (Methods).

The overlap sweep spans $R_B = 1$ on the T4. Holding residency fixed and scaling compute produced measured points on both sides of the derived boundary, between

Table 2 Descriptive residency comparison across 3 accelerators (layered probe, $d = 2048$). T is mean per-step latency; "Ratio" is capacity-limited over resident. The direction (ratio > 1) is the predicted effect; magnitudes are device-dependent and not predicted.

Accelerator	Resident T (ms)	Cap.-limited T (ms)	Ratio	Overlap sweep
Tesla T4	14.5	41.1	2.83×	sweep spans $R_B = 1$
NVIDIA A100	52.4	70.6	1.35×	sweep B compute-bound
Google TPU v5e	220.6	270.6	1.23×	R_B split unreliable (XLA)

$R_B = 0.90$ and $R_B = 1.19$. Their I/O-limited and coordination-dominated labels follow from the declared rule and therefore do not independently validate it; the run establishes physical reachability of both sides with separately timed services. On the A100 the controlled sweep stayed compute-bound ($R_B > 2$), and on the TPU the XLA path's deferred execution makes the per-operation R_B split unreliable; both need a larger-workload, preregistered run.

Discussion

The two-ratio description separates two questions that are often conflated in hierarchical-memory optimisation: whether the active data can reside in the fast tier, and whether compulsory transfer can be hidden behind computation. The resulting labels are operational rather than universal phases: $\theta_B = 1$ follows exactly from the overlap definition, whereas $\theta_C = 0.5$ is a declared midpoint that another application may replace without changing Equation (3).

The processor probes show that latency depends strongly on working-set residency but do not validate a sharp boundary at $R_C = 0.5$. The accelerator runs exercise real DMA and asynchronous CUDA streams on three devices of two vendors: the residency direction is consistent on all three, and the T4 sweep contains points on both sides of $\theta_B = 1$. The data do not establish per-device boundary *values*, HBM- or collective-communication effects at scale, or the generality of the three labels across model families.

The framework is most useful as a measurement discipline. Reporting R_C and R_B alongside latency makes hidden assumptions about residency and overlap inspectable, and prevents a common circularity: using total step duration both in a bandwidth ratio and as the outcome being explained. Its relevance is not tied to one model family. Weights, KV caches, embeddings, feature maps and retrieval indices are different objects, but each admits the same audit: declare the active bytes and usable fast tier, measure compulsory cross-tier bytes and sustained bandwidth, and isolate compute time. Equations (4) and (5) then state which part of the pressure cannot be removed by placement or ideal overlap. This creates a common reporting layer under otherwise incomparable offloading, compression, scheduling and memory-pooling studies. Whether it predicts application outcomes is a hypothesis for production workloads, not a result of the present probes.

The analysis supports one practical conclusion: capacity and transfer pressure should not be inferred from latency alone. A defensible comparison should report the

Table 3 Minimal reporting contract for a hierarchical-memory operating point. Reporting these fields makes the residency and overlap ratios recomputable and their denominator choices auditable. Continuous ratios are mandatory; categorical labels are optional and, when used, must name the residency convention θ_C.

Field	Symbol	Definition and measurement rule
Active working set	W	Bytes an execution step can touch; the denominator of R_C. State inclusion of weights, activations, and KV cache.
Usable fast capacity	C_{fast}	*Usable* fast-tier bytes, measured or from sysfs—not nominal die capacity.
Compulsory traffic	D	Cross-tier bytes that must move per step; denominator of T_{transfer}.
Non-resident bytes	D_{nr}	Compulsorily accessed non-resident bytes; needed to instantiate the lower bound.
Sustained bandwidth	B_{slow}	Achieved bandwidth on the limiting link under load—not datasheet peak.
Isolated compute time	T_{comp}	Step compute time with transfer disabled or separately timed.
Isolated transfer time	T_{transfer}	D/B_{slow}, timed independently of compute.
Residency ratio	R_C	C_{fast}/W, reported as a continuous value.
Overlap ratio	R_B	$T_{\text{comp}}/T_{\text{transfer}}$, reported as a continuous value.
End-to-end latency	T_{total}	Measured step latency, the outcome kept separate from the ratio denominators.
Residency convention	θ_C	Declared majority-residency cut-off if a categorical label is reported (default 0.5).

minimal field set in Table 3, which fixes the denominator of each ratio, separates a failed directional prediction from a merely small effect, allows threshold sensitivity to be recomputed under another residency convention, and lets an independent reader recompute (R_C, R_B) without the original latency trace. This reporting contract—not any single measured number—is the principal reusable output of the study.

Several limitations are material. The present evidence comes from one processor and three accelerators (two GPUs and a TPU) with small synthetic probes; it does not include production models, concurrent requests, energy measurements, policy comparisons or unseen-hardware classifier tests. The standard deviations describe repeated trials on one machine, not a population of platforms; the accelerator ratios compare repeated windows within single runs and quantify neither run-to-run uncertainty nor a hardware population; and no causal inference is supported. The pointer-chain endpoints span several hierarchy transitions, so their ratio is not a universal effect size. Finally, Equation (2) is an accounting identity only when its terms are defined as non-overlapping critical-path contributions; the max-form lower bound avoids assuming that decomposition in measured systems.

A decisive follow-up should preregister operating-point grids around $R_C = 0.5$ and $R_B = 1$, collect raw event traces on at least two accelerator architectures, compare fixed orchestration policies on production serving stacks without changing other workload parameters, and repeat the analysis across plausible residency conventions rather than tune a threshold on the evaluation data. Until then, the contribution is

the identifiable coordinate, its bounds and reporting contract, and a processor-and-multi-accelerator falsification attempt—not universal accelerator validation.

References

[1] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. AI and Memory Wall. *IEEE Micro*, 44(3):33–39, 2024. URL https://doi.org/10.1109/MM.2024.3373763.

[2] Xiaoyu Ma and David Patterson. Challenges and Research Directions for Large Language Model Inference Hardware. *Computer*, 59(5):55–64, 2026. URL https://doi.org/10.1109/MC.2026.3652916.

[3] International Energy Agency. Electricity 2024: Analysis and forecast to 2026. Technical report, IEA, 2024. URL https://www.iea.org/reports/electricity-2024. Global average carbon intensity: 494 gCO_2/kWh.

[4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023. URL https://doi.org/10.1145/3600006.3613165.

[5] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2303.06865.

[6] Chien-Chin Huang, Gu Jin, and Jinyang Li. SwapAdvisor: Pushing Deep Learning Beyond the GPU Memory Limit via Smart Swapping. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020. URL https://doi.org/10.1145/3373376.3378530.

[7] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Proc. NeurIPS*, volume 35, pages 16344–16359, 2022. URL https://arxiv.org/abs/2205.14135.

[8] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. In *Proc. HPCA*, pages 1348–1362, 2025. URL https://doi.org/10.1109/HPCA61900.2025.00102.

[9] Isabel Juniewicz. Hyperscaler capex is on trend to outpace their cash inflows by the end of 2026. Epoch AI, Data Insight, 2026. Accessed: 2026-06-21. [Online]. Available: https://epoch.ai/data-insights/hyperscaler-capex-vs-cash-flow.

[10] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL https://arxiv.org/abs/2407.21783.

[11] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024. URL https://arxiv.org/abs/2401.04088.

[12] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *Proc. ICLR*, 2021. URL https://arxiv.org/abs/2010.11929.

[13] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *Proc. ICML*, 2023. URL https://arxiv.org/abs/2301.12597.

[14] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. URL https://doi.org/10.1109/TBDATA.2019.2921572.

[15] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, Shengen Yan, Guohao Dai, Xiao-Ping Zhang, Yuhan Dong, and Yu Wang. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294*, 2024. URL https://arxiv.org/abs/2404.14294.

[16] Baolin Li et al. LLM inference serving: Survey of recent advances and opportunities. In *Proc. IEEE HPEC*, pages 1–8, 2024. URL https://doi.org/10.1109/HPEC62836.2024.10938426.

[17] Xupeng Miao et al. Towards efficient generative large language model serving: A survey from algorithms to systems. *ACM Computing Surveys*, 58(1):1–37, 2025. URL https://arxiv.org/abs/2312.15234.

[18] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *Proc. OSDI*, pages 521–538, 2022. URL https://www.usenix.org/conference/osdi22/presentation/yu.

[19] Kuntai Du et al. PrefillOnly: An inference engine for prefill-only workloads in large language model applications. In *Proc. SOSP*, pages 399–414, 2025. URL https://doi.org/10.1145/3731569.3764834.

[20] Yibo Jin et al. P/D-Serve: Serving disaggregated large language model at scale. *arXiv preprint arXiv:2408.08147*, 2024. URL https://arxiv.org/abs/2408.08147.

[21] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Chenyang Li, Dong Li, Erich Zheng, Olatunji Ruwase, and Yuxiong He. DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 1–15, 2022. URL https://doi.org/10.1109/SC41404.2022.00051.

[22] Yi Xu et al. Pie: Pooling CPU memory for LLM inference. *arXiv preprint arXiv:2411.09317*, 2024. URL https://arxiv.org/abs/2411.09317.

[23] Xuanlin Jiang et al. NEO: Saving GPU memory crisis with CPU offloading for online LLM inference. In *Proc. MLSys*, volume 7, 2025. URL https://proceedings.mlsys.org/paper_files/paper/2025/hash/66a026c0d17040889b50f0dfa650e5e0-Abstract-Conference.html.

[24] Xiangwen Zhuge et al. SpecOffload: Unlocking latent GPU capacity for LLM inference on resource-constrained devices. In *Proc. IEEE/ACM IWQoS*, pages 1–10, 2026. URL https://doi.org/10.1109/IWQoS70441.2026.11661278.

[25] Jiale Xu et al. eLLM: Elastic memory management framework for efficient LLM serving. *arXiv preprint arXiv:2506.15155*, 2025. URL https://arxiv.org/abs/2506.15155.

[26] Kun Wang et al. SwapNet: Efficient swapping for DNN inference on edge AI devices beyond the memory budget. *IEEE Transactions on Mobile Computing*, 23(9):8935–8950, 2024. URL https://arxiv.org/abs/2401.16757.

[27] Shun Kida et al. Revisiting memory swapping for big-memory applications. In *Proc. HPCAsia*, pages 33–42, 2025. URL https://doi.org/10.1145/3712031.3712332.

[28] Elias Frantar and Dan Alistarh. SparseGPT: Massive language models can be accurately pruned in one-shot. In *Proc. ICML*, 2023. URL https://arxiv.org/abs/2301.00774.

[29] Vijay Anand Korthikanti et al. Reducing activation recomputation in large transformer models. *Proc. MLSys*, 5:341–353, 2023. URL https://arxiv.org/abs/2205.05198.

[30] Yanping Huang et al. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Proc. NeurIPS*, 2019. URL https://arxiv.org/abs/1811.06965.

[31] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019. URL https://arxiv.org/abs/1909.08053.

[32] A.H. Hunter et al. Beyond malloc efficiency to fleet efficiency: a hugepage-aware memory allocator. In *Proc. OSDI*, 2021. URL https://www.usenix.org/conference/osdi21/presentation/hunter.

[33] Hanmei Yang et al. NUMAlloc: A faster NUMA memory allocator. In *Proc. ISMM*, pages 97–110, 2023. URL https://doi.org/10.1145/3591195.3595276.

[34] Ugljesa Milic, Oreste Villa, Evgeny Bolotin, Akhil Arunkumar, Eiman Ebrahimi, Aamer Jaleel, Alex Ramirez, and David Nellans. Beyond the socket: NUMA-aware GPUs. In *Proc. MICRO*, pages 123–135, 2017. URL https://doi.org/10.1145/3123939.3124534.

[35] Carl A. Waldspurger. Memory resource management in VMware ESX Server. In *Proceedings of the 5th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 181–194, 2002. URL https://www.usenix.org/conference/osdi-02/memory-resource-management-vmware-esx-server.

[36] Sifan Long et al. A survey on intelligent network operations and performance optimization based on large language models. *IEEE Communications Surveys & Tutorials*, 27(6):3915–3949, 2025. URL https://doi.org/10.1109/COMST.2025.3526606.

[37] Xiaofei Wang, Yiwen Han, Victor C. M. Leung, Dusit Niyato, Xueqiang Yan, and Xu Chen. Convergence of edge computing and deep learning: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 22(2):869–904, 2020. URL

https://doi.org/10.1109/COMST.2020.2970550.
[38] Rishov Sarkar et al. Edge-MoE: Memory-efficient multi-task vision transformer architecture with task-level sparsity via mixture-of-experts. In *Proc. ICCAD*, pages 1–9, 2023. URL https://doi.org/10.1109/ICCAD57390.2023.10323651.
[39] Haoxiang Luo et al. Toward edge general intelligence with multiple-large language model (Multi-LLM): architecture, trust, and orchestration. *IEEE Trans. Cogn. Commun. Netw.*, 11(6):3563–3585, 2025. URL https://doi.org/10.1109/TCCN.2025.3612760.
[40] Zixuan He et al. CPU–GPU heterogeneous computation offloading and resource allocation scheme for industrial Internet of Things. *IEEE Internet of Things Journal*, 11(6):11152–11164, 2024. URL https://ieeexplore.ieee.org/document/10319405.
[41] Olivier Beaumont et al. Optimal GPU-CPU offloading strategies for deep neural network training. In *Proc. Euro-Par*, pages 151–166, 2020. URL https://doi.org/10.1007/978-3-030-57675-2_10.
[42] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger. Evaluation techniques for storage hierarchies. *IBM Systems Journal*, 9(2):78–117, 1970. doi: 10.1147/sj.92.0078. URL https://doi.org/10.1147/sj.92.0078.
[43] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785. URL https://doi.org/10.1145/1498765.1498785.
[44] Leonard Kleinrock. *Queueing Systems, Volume 1: Theory.* John Wiley & Sons, New York, 1975. ISBN 978-0-471-49110-1.

Methods

Definitions and classification

We calculate $R_C = C_{\text{fast}}/W$ from a declared usable fast-tier capacity and active working-set size. We calculate $R_B = T_{\text{comp}}/T_{\text{transfer}}$, with compute and transfer timed separately. Classification gives capacity precedence: $R_C < 0.5$ is capacity-limited; otherwise $R_B < 1$ is I/O-limited; all remaining points are coordination-dominated. This rule is implemented in the released Python code. We retain the continuous ratios in every record because 0.5 is a reporting convention rather than an estimated device boundary. Any reuse should declare W and usable C_{fast}, report the unthresholded ratios, and repeat categorical summaries under application-relevant residency cut-offs when the conclusion depends on the label.

Pointer-chain capacity probe

The CPU probe allocates a NumPy array and constructs a seeded random permutation. A C11 kernel compiled with `-O3` traverses it as a dependent pointer chain; each loaded index determines the address of the next load, limiting vectorisation, instruction-level parallelism, and prefetching. We measured working sets of 0.5, 1, 2, 4,

8, 16, 24, 32, 48, 64, 96, 128, 192, and 256 MiB. Each point used seven timed trials after warm-up. The output stores working-set size, R_C, mean nanoseconds per dependent load, standard deviation, and trial count. The fast-tier capacity was read from Linux sysfs and was 48 MiB. The host exposed three logical CPUs of an Intel Xeon Platinum 8370C under KVM; the summary records the kernel, Python, and NumPy versions. Timing used `clock_gettime(CLOCK_MONOTONIC_RAW)` inside the compiled loop. No outlier was removed and CPU affinity or frequency was not controlled.

Layered matrix and copy probe

The second probe allocates square, float32 layer matrices; a selected fraction remains resident and non-resident matrices are copied before multiplication. Matrix entries are deterministically seeded and scaled by $1/\sqrt{d}$ to prevent numerical overflow during repeated multiplication. On the CPU we used NumPy's wall-clock path with 12 layers, $d = 512$, batch size 8, and five windows per point. The mean total latency averaged over the three sampled points below $R_C = 0.5$ was 4.03 ms, compared with 1.87 ms for the fully resident point, a 2.15-fold descriptive ratio. Because computation and copying share the same processor memory system, the accompanying R_B sweep was noisy and is not used as evidence for the I/O boundary. On the accelerators the same program ran with 24 layers, $d = 2048$, batch size 8, ten windows, and pinned host staging: on the NVIDIA Tesla T4 and A100 through the harness's CUDA-event path, timing computation and host-to-device transfer separately with `torch.cuda.Event`; and on a Google TPU v5e through the `torch_xla` path. Because XLA executes lazily, per-op R_B timing on the TPU is unreliable and we treat its R_B split as indicative only, relying on end-to-end latency for the residency test.

Fully resident point on fast devices

Latency is monotone in the number of offloaded layers down to two offloaded layers on all three accelerators; the only departure is the single fully resident point ($R_C = 1$, zero transfers), which sits above the trend on the A100 (+9 ms) and the TPU (+42 ms) but not on the slower T4. This is the one operating point at which the offload code path is skipped entirely, so it executes a structurally distinct computation—on the TPU a separately compiled XLA graph. At $d = 2048$ the true per-step memory signal is small on the fast accelerators, so this point's one-off compilation and launch overhead exceed the residual signal, whereas on the slower T4 the signal dominates and monotonicity is preserved. An eager-execution control (NumPy backend, identical sweep) shows the fully resident point as the *lowest* latency. This control is consistent with an accelerator execution/timing artifact but does not isolate a single cause. A discarded warm-up iteration has since been added to the harness so that per-point compilation is excluded from future timings; the reported accelerator runs were not repeated with it.

For the reported capacity ratio, we averaged total latency at the three sampled points below $R_C = 0.5$ and divided by latency at $R_C = 1$. This endpoint summary is descriptive; no hypothesis test or population-level confidence interval is claimed. Accelerator timing windows are repeated observations within one run, not independent

device or run replicates. The pointer-chain table reports mean and population standard deviation across the seven repeated trials exactly as stored in the JSON summary.

Software and provenance

The compiled pointer-chain run used Python 3.12.13, NumPy 2.5.2, and GCC 13.3; the earlier layered-matrix harness check used Python 3.11.15, but that summary did not capture the NumPy version. Seeds, grids, timing code, CPU raw records, and all reported summaries are version controlled. Accelerator JSON files retain per-point summaries but not raw device-event timestamps. Reproduction commands are version controlled. The analytical regime map reads threshold constants from the same module as the classifier. The simulated backend is provided solely for software testing and qualitative exploration; its outputs are excluded from the reported measurements.

Statistics and reporting

This proof-of-concept study did not use random assignment, blinding, or a sample-size calculation. Trial counts were fixed before summarisation. We report all measured grid points in the machine-readable summaries, selected points in Table 1, arithmetic means, and one standard deviation. For the selected pointer-chain points we additionally report a 95% percentile bootstrap confidence interval for the mean (10,000 resamples of the seven committed per-trial latencies, fixed seed for reproducibility). This interval describes within-machine trial variability only; because $n = 1$ machine, it is not a confidence interval over a hardware population, and no accelerator interval is reported because per-window raw records were not retained from the accelerator runs. No claim of statistical significance is made. Hardware-platform generalisation requires independent machines and is left for future work.

Reproducibility boundary

The repository contains the CPU raw JSONL records, per-accelerator JSON summaries, machine-readable aggregate summaries, and the export scripts that generate Tables 1 and 2 and the numerical macros in the text. It also contains a simulated backend for testing the analysis pipeline; simulated output is labelled as such and is not used as evidence. The accelerator summaries support only the proof-of-concept comparisons reported here and are not a substitute for raw event traces or independent hardware replication.

Data availability

All data supporting the reported CPU and accelerator measurements are included in the public repository at https://github.com/leemgs/orion, under `code/results/cpu_probe/` and `code/results/colab_probe/`. These directories contain the raw JSONL records and machine-readable JSON summaries used in the text and tables.

Code availability

The analysis and measurement code is available at https://github.com/leemgs/orion. The `code/experiments/` directory contains the CPU probes, the analytical-figure generator, the optional CUDA/XLA measurement paths, and the preregistered analysis. Exact reproduction commands are provided in the README and Supplementary Information. A separately labelled simulated backend is supplied for software testing; simulated output is not used as evidence in this article.

Acknowledgements

We thank Donghyeon Kang, Seungmin Oh and Yunsu Lee for valuable feedback. We are also grateful to Wooyeon Lee, Hyoyoung Kim, and Heekuk Lee, experts in systems and memory, whose insights and meaningful comments greatly improved this work.

Author contributions

G.L.: Conceptualization, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing.

Competing interests

The author declares no competing interests.